Snackautomat

Thema: Java

Dokumentinformationen

Dateiname:	Dokument3
Speicherdatum:	17.04.2026

Autoreninformationen

Autor: Nico Schult
Kara Diggelmann
Kaylin Vang

Inhaltsverzeichnis

1 Informieren 3
2 Planen 3
3 Entscheiden 3
4 Realisieren 3
5 Kontrollieren 4
6 Auswerten 4
7 Fazit 4

IPERKA

1 Informieren

Diese Phase ist ausschlaggebend für den Erfolg unseres Projekts. Die Struktur, die Programmiersprache, unsere Tools, erste Vorschläge für unsere Programmstruktur, allfällige Schwierigkeiten und weitere Punkte gehören dazu. Eilt unser Team zu schnell in die Realisierung, geht langfristig deutlich mehr Zeit verloren bei Korrekturen und Planänderungen als gespart würden durch eine Verkürzung der Planungsphase.

Zunächst mussten wir klarstellen, wie wir bis 10:00 Uhr vorgehen und wie wir unsere ersten Aufgaben bis 12:15 aufteilen sollen. Da ab 10:15 die Deepwork-Phase ohne Teams-Kommunikation oder physischer Austausch beginnen sollte, mussten wir eine Balance zwischen klaren Instruktionen und optionalen Aufgaben erstellen, sodass wir mangels Informationsflusses weiterhin in dieselbe Richtung ziehen können.

Somit teilten wir zuerst die Verfassung des UML/Flowcharts, der Aufteilung der Java-Klassen und die Strukturierung der Dokumentation anhand von IPERKA unter uns auf. Da wir wussten, dass der Arbeitsaufwand nicht komplett vergleichbar war, erlaubten wir es uns schon während der Deepwork-Phase bereits am Code zu arbeiten.

2 Planen

Wir wussten, dass wir Mittwoch, Donnerstag und noch etwa eine Stunde am Freitagmorgen zur Verfügung hatten. Zudem war uns bewusst, dass die Dokumentation und Planung schwerer gewichtet sein wird im Verhältnis zum Code verglichen mit anderen Projekten. Somit erlaubten wir uns genügend Zeit für die Phasen "Informieren", "Planen" und "Entscheiden vor am Mittwochmorgen. Wir nahmen auch ähnliche Projekte als Vorbild online, um unseren Zeitaufwand einschätzen zu können. Mittwochnachmittag erstellen wir Branches auf unserem Github-Repository, um Merge-Konflikte bei unserem Code frühzeitig vorzubeugen.

Mittwochnachmittag wollten wir bereits das Grundgerüst unserer Klassen-Struktur abgeschlossen haben inklusive erste Dokumentierung. Erste Ideen für die Präsentation am Freitag besprachen wir auch schon. Optionen bezüglich GUI für unser Programm schnitten wir ebenso an, würden jedoch frühestens am Donnerstagmorgen implementiert werden.

Donnerstagnachmittag würden Feinschliffe für die Funktionalität des Programms und Testing anstehen, zudem unsere Präsentation üben und unsere Dokumentation visuell verschönern.

3 Entscheiden

Wir entschieden uns, unsere Programmstruktur simpel zu halten, ohne zu grosse Inventare für den Snackautomaten oder komplexe finanzielle Abläufe. Unsere Präsentation würde zur Hälfte unser Vorgehen darstellen und zur Hälfte eine Demo unseres Programms sein. Diese Gewichtung ist ausgeglichen und zugleich flexibel.

4 Realisieren

Wir arbeiteten bezüglich des Codes so gut wie möglich testgetrieben. Dies wird durch die Aufteilung in Branches erleichtert, denn so kann jede Person ihren Code regelmässig in Isolation testen und so früh genug Fehler erkennen. Dieses Testen wäre zunächst statisch. So können

Fehler ausgebügelt werden, bevor das dynamische Testen beginnt. So können wir allfällige Fehlerquellen früh genug unter Kontrolle behalten.

Während der Realisierung kam es kaum zu Merge-Konflikten, da wir auf separaten Branches und separaten Klassen arbeiteten, somit verlieft unser Plan geschmeidig. Dies gab uns genug Zeit, zusätzliche UI-Features mit JOptionPane einzufügen inklusive Bilder, welches ein schönes Interface für den Snackautomaten ermöglichte.

Wir arbeiteten phasenweise, um eine Balance zwischen Trennung und Merges zu erlauben. So konnten wir ein Code-Chaos verhindern aber dennoch regelmässig eine einheitliche Grundlage unseres Programms aufrechterhalten.

5 Kontrollieren

Wir testeten regelmässig das ganze Programm und kommunizierten auf Teams allfällige Probleme. Dies geschah im Durchschnitt stündlich, je nach Phase der Entwicklung.

6 Auswerten

Rückblickend war unser Vorgehen insgesamt erfolgreich. Die frühe Aufteilung der Aufgaben, die klare Strukturierung nach IPERKA und die Arbeit mit Branches halfen uns dabei, effizient und organisiert zu bleiben. Besonders positiv war, dass wir trotz der Deepwork-Phase ohne direkten Austausch weiterhin am gleichen Ziel arbeiteten und kaum Missverständnisse entstanden.

Ebenfalls bewährte sich unsere Entscheidung, die Programmstruktur eher einfach zu halten. Dadurch konnten wir die wichtigsten Funktionen zuverlässig umsetzen, anstatt uns in unnötiger Komplexität zu verlieren. Die regelmässigen Tests und die laufende Kommunikation führten dazu, dass Probleme früh entdeckt und behoben werden konnten.

Verbesserungspotenzial sehen wir vor allem bei der noch genaueren Zeitplanung einzelner Arbeitspakete. Teilweise zeigte sich, dass gewisse Aufgaben mehr Zeit beanspruchten als ursprünglich angenommen. Für ein zukünftiges Projekt würden wir deshalb noch klarere Zwischenziele und Zeitpuffer einplanen.

7 Fazit

Insgesamt war das Projekt ein Erfolg. Wir konnten einen funktionsfähigen Snackautomaten entwickeln, der zentrale Anforderungen wie Käufe, Kaufabbrüche, finanzielle Abläufe, Inventarverwaltung und Nachbestellungen abdecken konnte. Gleichzeitig gelang es uns, die Arbeit im Team gut zu organisieren und unsere Dokumentation sinnvoll nach IPERKA aufzubauen.

Besonders wichtig für den Erfolg waren die gründliche Vorbereitung, die saubere Aufgabenverteilung, die Arbeit mit GitHub-Branches und das regelmässige Testen. Das Projekt zeigte uns, wie wertvoll eine gute Planung und klare Kommunikation in der Softwareentwicklung sind. Für zukünftige Gruppenarbeiten nehmen wir vor allem mit, dass eine einfache, gut durchdachte Lösung oft effizienter ist als ein unnötig komplexer Ansatz.